

Project 1 Final Report

CSE 490 Spring 2026

University at Buffalo

Marco Bianco - 50450154

Krish Puwar - krishpuw - 50453913

Andrew Collado – arcollad - 50401294

Project Overview

For this project we designed and implemented a simple 16-bit non-pipelined processor in Verilog and ran it on a Xilinx Basys 3 FPGA board. The processor is single-cycle, meaning every instruction completes in one clock cycle from fetch to write-back with no pipelining. It has a 16-bit word size, byte-addressable memory, 16 general-purpose registers, and supports 10 instructions split across three formats: R-type, I-type, and J-type. We used Xilinx Vivado 2025.2 for simulation, synthesis, and bitstream generation.

The design is made up of 10 separate Verilog modules that are wired together at the top level using structural Verilog. Each module handles one piece of the processor, and the top-level structural file just connects them all. The individual modules are written in behavioral Verilog.

Work Distribution

Group Member	Assigned Components	Other Contributions
Marco Bianco, marcobia	ALU, multiplexer, data memory	testbench, simulation
Krish Puwar,	Sign extension, Control Unit, Program counter	program.mem, debugging
Andrew Collado, arcollad	Register file, instruction memory	Final report

Program Counter (pc) -- program_counter.v

The program counter is a 16-bit register that keeps track of which instruction to fetch next. On every rising clock edge, it updates to the next PC value that the datapath feeds into it. That next value is one of three things: PC+2 for a normal sequential step, a branch target, or a jump target. The +2 comes from the fact that each instruction is 2 bytes wide, and memory is byte-addressed.

```
module pc(
    input wire clk,
    input wire reset ,
    output reg[15:0] outputpc,
    input wire[15:0] inputpc
);

    always @(posedge clk)
    begin
        if (reset) begin
            outputpc <= 16'h0000;
        end
        else
            outputpc <= inputpc +16'd2; // pc+2
        end
    end

endmodule
```

Instruction Memory - instruction_memory.v

The instruction memory is a byte-addressed array with 128 entries. Since each instruction is 16 bits, the module reads two adjacent bytes and concatenates them to form the full instruction word. It is initialized from a file called program.mem using \$readmemh, which loads each byte as a hex value. The PC value coming in is used as the index into this array.

```
module instruction_memory (
    input wire [15:0] address, // byte addr from pc
    output wire [15:0] instruction
);

    // memory array is 256 bytes and instruct is at EVEN addresses (each spans 2 bytes)
    reg [7:0] mem [0:255];

    initial begin
        // program.mem is just a placeholder located at project_1.sim/sim_1/behav/xsim/
        $readmemh("program.mem", mem);
    end

    // instructions stored as big-endian, heres an example
    // addr 0x00: addi $s1, $s0, 5 (16'h3105)
    // mem[8'h00] = 8'h31; mem[8'h01] = 8'h05;

    // async read - fetch two bytes and combine big-endian, address must be word-aligned (even) and
    // address[0] is ignored
    assign instruction = {mem[address], mem[address + 16'd1]};

endmodule
```

Register File - register_file.v

The register file holds 16 general-purpose 16-bit registers, \$s0 through \$s15. It has two read ports, and one write port. The reads are asynchronous, so the output changes as soon as the register indices change. The write is synchronous and only happens on a rising clock edge when the RegWrite control signal is asserted.

```
module register_file (
    input wire      clk,
    input wire      reset,

    // read 1 - rs field [7:4]
    input wire [3:0] read_reg1,    // register the index to read
    output wire [15:0] read_data1,  // data read from rs

    // read 2 - rt or rd field [11:8]
    input wire [3:0] read_reg2,    // same as above
    output wire [15:0] read_data2,  // same as above

    // write
    input wire [3:0] write_reg,     // reg index to write to
    input wire [15:0] write_data,   // the data to write
    input wire      reg_write       // write enable activ-high
);

// 16 registers, each 16 bits wide
reg [15:0] registers [0:15];
integer i;

// synchronous write / reset
always @(posedge clk) begin
    if (reset) begin
        // zero out all of the reg on reset
        for (i = 0; i < 16; i = i + 1)
            registers[i] <= 16'b0;
        // otherwise start write the data
    end else if (reg_write) begin
        registers[write_reg] <= write_data;
    end
end

// async read perchanace
assign read_data1 = registers[read_reg1];
assign read_data2 = registers[read_reg2];

endmodule
```

ALU - ALU.v

The ALU takes two 16-bit inputs and a 3-bit control signal and produces a 16-bit result. It also outputs a zero flag that goes high whenever the result is zero, which the branch instructions rely on. The four operations it supports are addition (000), subtraction (001), logical shift left (010), and bitwise AND (011). Carry and overflow are not handled.

```
module ALU(
    input [15:0] a,
    input [15:0] b,
    input [2:0] alu_control,
    output reg zero,
    output reg [15:0] result
);
//0 - add
//1 - sub
```

```

//2 - sll
//3 - and
//else result == 0

always@(*)
begin
    if (alu_control == 0) begin
        result = a + b;
    end
    else if(alu_control == 1) begin
        result = a - b;
    end
    else if(alu_control == 2) begin
        result = a << b[3:0];
    end
    else if(alu_control == 3) begin
        result = a & b;
    end
    else begin
        result = 0;
    end

    if (result == 0) begin
        zero = 1;
    end
    else begin
        zero = 0;
    end
end
endmodule

```

ALU Control - alu_control.v

The ALU control module takes in a 2-bit ALUOp signal from the main control unit and the 4-bit function code from the instruction, and outputs the 3-bit operation code that the ALU uses. For I-type instructions, the ALUOp directly determines the operation. For R-type instructions it looks at the function code to decide whether to add, subtract, shift, or AND.

```

module alu_control(
    input wire [1:0] aluop,
    input wire [2:0] funct,
    output reg [2:0] alu_control
);

always @(*) begin
    case (aluop)
        2'b00: alu_control = 3'b000; // add

        2'b01: alu_control = 3'b001; // sub

        2'b10: begin // r type
            case (funct)
                3'b000: alu_control = 3'b000; // add
                3'b001: alu_control = 3'b001; // sub
                3'b010: alu_control = 3'b010; // sll
                3'b011: alu_control = 3'b011; // and
                default: alu_control = 3'b000;
            endcase
        end

        default: alu_control = 3'b000;
    endcase
end
endmodule

```

Sign Extension - sign_extension.v

The sign extension module takes the small immediate field from an instruction and extends it to 16 bits while preserving the sign. For I-type instructions the immediate is 4 bits, so the module replicates bit 3 into the upper 12 bits. For the J-type jmp instruction, the address field is 12 bits, so it gets extended to 16 bits by replicating bit 11.

```
module sign_extension (  
    input wire[3:0] imm,   
    output wire[15:0] sign_extend_imm  
);  
  
    assign sign_extend_imm = {{12{imm[3]}}, imm};  
  
endmodule
```

Multiplexer - mux.v

The multiplexer is a 2-to-1 16-bit mux. Multiple instances of it are used in the datapath. One selects whether the second ALU input comes from a register or from the sign-extended immediate, controlled by the ALUSrc signal. Another selects whether the value written back to the register file comes from the ALU result or from data memory, controlled by MemToReg. Additional instances handle the PC update logic for branches and jumps.

```
module Multiplexer(  
    input [15:0] a,  
    input [15:0] b,  
    input select,  
    output reg [15:0] out  
);  
always@(a,b,select)  
begin  
    if(select == 1)begin  
        out = b;  
    end  
    else begin  
        out = a;  
    end  
end  
end  
endmodule
```

Data Memory - data_memory.v

The data memory is also a 128-entry byte-addressed array. Words are stored in big-endian order, meaning the high byte is at the lower address. Reads are asynchronous when MemRead is asserted, and writes are synchronous on the rising clock edge when MemWrite is asserted. The sw instruction triggers a write and lw triggers a read. The memory is initialized with default values in the initial block.

```
module data_memory(  
    input clk,  
    input [15:0] address,  
    input [15:0] writedata,  
    input memwrite,
```

```

input memread,
output reg [15:0] readdata
);

reg [7:0] memory [0:255];
always @(posedge clk) begin
if (memwrite) begin
    memory[address] <= writedata[15:8];
    memory[address + 1] <= writedata[7:0];
end
end
always @(*) begin
if (memread)
    readdata = {memory[address], memory[address + 1]};
else
    readdata = 16'b0;
end
endmodule

```

Control Unit - control_unit.v

The control unit is a combinational module that looks at the 4-bit opcode and sets all the control signals for the rest of the datapath. It uses a case statement to drive RegWrite, MemRead, MemWrite, MemToReg, ALUSrc, ALUOp, Branch, BranchNE, and Jump. For R-type instructions where the opcode is 0000, it also passes the function code along to the ALU control module. For add it gates the RegWrite signal based on whether the condition register is zero.

```

`timescale 1ns / 1ps
module control_unit (

    input wire[3:0] opcode,
    input wire[2:0] func,
    output reg regdst,
    output reg alusrc,
    output reg MemtoReg,
    output reg regtomem,
    output reg memwrite,
    output reg regwrite,
    output reg memread,
    output reg branch,
    output reg [1:0] aluop,
    output reg jump,
    output reg bne

);

always @(*) begin
    regdst = 1'b0;
    alusrc = 1'b0;
    MemtoReg = 1'b0;
    regtomem = 1'b0;
    memwrite = 1'b0;
    regwrite = 1'b0;
    memread = 1'b0;
    branch = 1'b0;
    aluop = 2'b00;
    jump = 1'b0;

    case (opcode)

        // R type
        4'b0000: begin

```

```

        regdst = 1'b1;
        alusrc = 1'b0;
        Memtoreg = 1'b0;
        regtomem = 1'b0;
        memwrite = 1'b0;
        regwrite = 1'b1;
        memread = 1'b0;
        branch = 1'b0;
        aluop = 2'b10;
        jump = 1'b0;
        bne = 1'b0;

end

// lw
4'b0001: begin
    regdst = 1'b0;
    alusrc = 1'b1;
    Memtoreg = 1'b1;
    regtomem = 1'b0;
    memwrite = 1'b0;
    regwrite = 1'b1;
    memread = 1'b1;
    branch = 1'b0;
    aluop = 2'b00;
    jump = 1'b0;
    bne = 1'b0;

end

//sw
4'b0010: begin
    regdst = 1'b0;
    alusrc = 1'b1;
    Memtoreg = 1'b0;
    regtomem = 1'b1;
    memwrite = 1'b1;
    regwrite = 1'b0;
    memread = 1'b0;
    branch = 1'b0;
    aluop = 2'b00;
    jump = 1'b0;
    bne = 1'b0;

end

// addi
4'b0011: begin
    regdst = 1'b0;
    alusrc = 1'b1;
    Memtoreg = 1'b0;
    regtomem = 1'b0;
    memwrite = 1'b0;
    regwrite = 1'b1;
    memread = 1'b0;
    branch = 1'b0;
    aluop = 2'b00;
    jump = 1'b0;
    bne = 1'b0;

end

//beq
4'b0100: begin
    regdst = 1'b0;
    alusrc = 1'b0;
    Memtoreg = 1'b0;
    regtomem = 1'b0;
    memwrite = 1'b0;
    regwrite = 1'b0;
    memread = 1'b0;

```

```

        branch = 1'b1;
        bne = 1'b0;
        // need to check for aluop
        aluop = 2'b01;
        jump = 1'b0;

    end
    // bne
    4'b0101: begin
        regdst = 1'b0;
        alusrc = 1'b0;
        Memtoreg = 1'b0;
        regtomem = 1'b0;
        memwrite = 1'b0;
        regwrite = 1'b0;
        memread = 1'b0;
        branch = 1'b1;
        bne = 1'b1;
        // aluop possibly
        aluop = 2'b01;
        jump = 1'b0;
    end
    // jump
    4'b0110: begin
        regdst = 1'b0;
        alusrc = 1'b0;
        Memtoreg = 1'b0;
        regtomem = 1'b0;
        memwrite = 1'b0;
        regwrite = 1'b0;
        memread = 1'b0;
        branch = 1'b0;
        // aluop possibly
        aluop = 2'b01;
        jump = 1'b1;
        bne = 1'b0;
    end
    default: begin
        regdst = 1'b0;
        alusrc = 1'b0;
        Memtoreg = 1'b0;
        regtomem = 1'b0;
        memwrite = 1'b0;
        regwrite = 1'b0;
        memread = 1'b0;
        branch = 1'b0;
        aluop = 2'b00;
        jump = 1'b0;
        bne = 1'b0;
    end
end
endcase

end

endmodule

```

Top-Level Structural Module - structural.v

The structural module is the top level of the design. It instantiates all nine other modules and connects them with wires. The only logic at this level is the PC+2 adder. Everything else is just port connections between modules. The external ports are just clock and reset. Internally the main signals are pc_out, the fetched instruction, the two register read values,

alu_result, mem_read_data, write_data, the sign-extended immediate, and the various control signal wires.

```
module structural(input clk, input reset);

    wire [15:0] program_count;
    wire [15:0] instruction;
    wire [15:0] read_data1;
    wire [15:0] read_data2;
    wire [15:0] alu_result;
    wire [15:0] write_data;
    wire [15:0] sign_extend;
    wire [15:0] alu;
    wire [15:0] mux5_input;
    wire [15:0] program_counter_input;
    wire [15:0] mem_data;
    wire [15:0] read_data;
    wire [15:0] pc_p2;
    wire [15:0] branch_a;
    wire [2:0] funct;
    wire reg_write, mem_read, mem_write, reg_dest, zero, regtomem;
    wire alu_src, mem_to_reg, branch, jump, bne;
    wire [2:0] alu_control;
    wire [3:0] write_reg;
    wire [1:0] aluop;
    wire [15:0] pc_2;

    assign pc_p2 = program_count;
    assign branch_a = pc_p2 + (sign_extend << 1);
    assign write_reg = instruction[11:8];

    pc PC(
        .clk(clk),
        .outputpc(program_count),
        .reset(reset),
        .inputpc(program_counter_input)
    );

    instruction_memory IM(
        .instruction(instruction),
        .address(program_count)
    );

    control_unit CU(
        .opcode(instruction[15:12]),
        .regdst(reg_dest),
        .alusrc(alu_src),
        .Memtoereg(mem_to_reg),
        .memwrite(mem_write),
        .regwrite(reg_write),
        .memread(mem_read),
        .branch(branch),
        .aluop(aluop),
        .jump(jump),
        .func(instruction[2:0])
    );

    register_file RF(
        .clk(clk),
        .reset(reset),
        .read_reg1(instruction[7:4]),
        .read_data1(read_data1),
        .read_reg2(instruction[11:8]),
        .read_data2(read_data2),
        .write_reg(write_reg),
        .write_data(write_data),
        .reg_write(reg_write)
    );

endmodule
```

```

sign_extension SE(
    .immd(instruction[3:0]),
    .sign_extend_imm(sign_extend)
);

//alu mux
Multiplexer MX1(
    .a(read_data2),
    .b(sign_extend),
    .select(alu_src),
    .out(alu)
);

// data mem mux
Multiplexer MX2(
    .a(alu_result),
    .b(mem_data),
    .select(mem_to_reg),
    .out(write_data)
);

// register file mux
//Multiplexer MX3(
//    .a(instruction[8:6]),
//    .b(instruction[5:3]),
//    .select(reg_dest),
//    .out(write_reg)
//);

// branch mux
Multiplexer MX4(
    .a(pc_p2),
    .b(branch_a),
    .select((branch & zero) | (bne & ~zero)),
    .out(mux5_input)
);

// jump mux
Multiplexer MX5(
    .a(mux5_input),
    .b(pc_p2 + {{3{instruction[11]}}, instruction[11:0], 1'b0}),
    .select(jump),
    .out(program_counter_input)
);

ALU AU(
    .a(read_data1),
    .b(alu),
    .alu_control(alu_control),
    .zero(zero),
    .result(alu_result)
);

data_memory DM(
    .clk(clk),
    .address(alu_result),
    .writedata(read_data2),
    .memwrite(mem_write),
    .memread(mem_read),
    .readdata(mem_data)
);

alu_control ALUCTL (
    .aluop(aluop),
    .funct(instruction[2:0]),
    .alu_control(alu_control)
);
endmodule

```

Top-Level Structural Test Bench - structural_tb.v

The structural test bench displays all our important modules/wiring. It displays time, program counter, instruction, read_data1/2, alu, alu_result, write_data, reg_write, sign_extend, mux_alu, mux_mem, and zero flag. It is connected to the structural clk and reset and gets toggled every 5 ns. The simulation runs for 1000 ns while continuously monitoring for signal changes.

```
`timescale 1ns / 1ps

module structural_tb;
    reg clk;
    reg reset;

    structural DUT(
        .clk(clk),
        .reset(reset)
    );

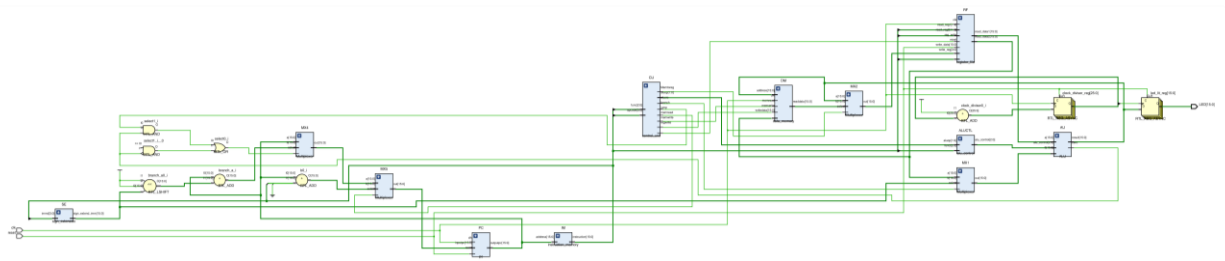
    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        reset = 1;
        #10;
        reset = 0;
        #1000;
        $finish;
    end

    initial begin
        $display("Time | PC | instr | r1 | r2 | alu_in2 | ALU_result | write_data | reg_write |
sign_extend | mux_alu | mux_mem | zero");
        $monitor("%0t | %d | %b | %d | %d | %d | %d | %d | %b | %d | %d | %d | %b",
            $time,
            DUT.program_count,
            DUT.instruction,
            DUT.read_data1,
            DUT.read_data2,
            DUT.alu,
            DUT.alu_result,
            DUT.write_data,
            DUT.reg_write,
            DUT.sign_extend,
            DUT.alu,
            DUT.write_data,
            DUT.zero
        );
    end
endmodule
```

Datapath and Control Unit Schematic

The datapath follows a standard single-cycle organization. The PC feeds into instruction memory, which outputs the current instruction. The control unit reads the opcode and drives signals throughout the rest of the datapath. The register file reads two source operands. A mux selects between the second register operand and the sign-extended immediate as the second ALU input, depending on whether the instruction is R-type or I-type. The ALU performs the operation and produces a result and a zero flag. For load and store, the ALU result is used as the memory address. For branch instructions, the zero flag determines whether the PC updates the branch target or PC+2. For jump, the PC always takes the jump target. The write-back mux chooses between the ALU result and data memory output before writing to the register file.



The control unit receives the opcode and function code and drives the following signals for each instruction:

Instruction	RegWrite	MemRead	MemWrite	MemToReg	ALUSrc	Branch/Jump
add/sub/sll/and	1	0	0	0	0	None
addi / addif	1	0	0	0	1	None
lw	1	1	0	1	1	None
sw	0	0	1	X	1	None
beq	0	0	0	0	0	Branch if zero=1
bne	0	0	0	0	0	Branch if zero=0
jmp	0	0	0	X	X	Always jump

Simulation Results

All simulations were run using the Vivado 2025.2 behavioral simulator (xsim). The testbench drives a 10 ns clock and runs for 210 ns total, which is enough to step through the full test program. The \$monitor statement prints key signals every cycle: PC, instruction bits, both register read values, the second ALU input, the ALU result, the write-back data, and the RegWrite flag.

Test Program

The test program is loaded from program.mem and exercises all instruction types. Below is the decoded version of each instruction along with what it should compute.

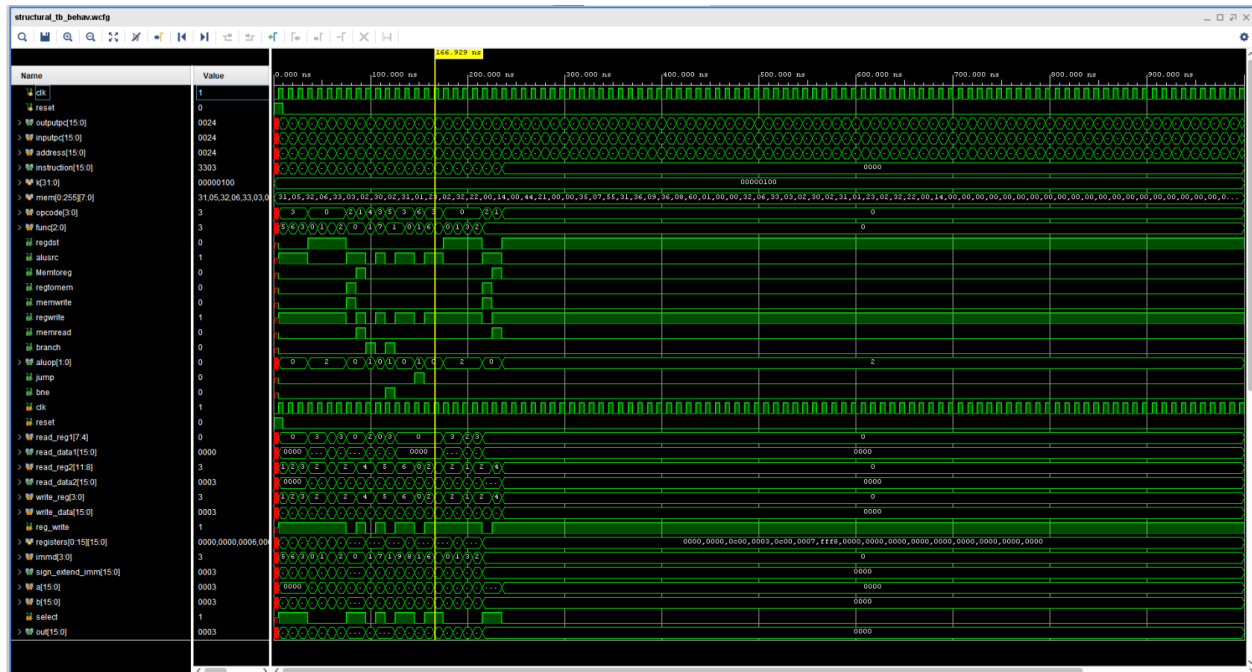
PC	Hex	Assembly	Expected result
0	31 05	addi \$s1, \$s0, 5	R1 = 5
2	32 06	addi \$s2, \$s0, 6	R2 = 6
4	33 03	addi \$s3, \$s0, 3	R3 = 3
6	02 30	add \$s2, \$s3	R2 = R3 + R2 = 9
8	02 31	sub \$s2, \$s3	R2 = R3 - R2 = -6 (65530 unsigned)
10	01 23	and \$s1, \$s2	R1 = R2 & R1 = 0
12	02 32	sll \$s2, \$s3	R2 = R2 << R3 = 3072
14	22 00	sw \$s2, 0(\$s0)	Mem[0] = 3072
16	14 00	lw \$s4, 0(\$s0)	R4 = Mem[0] = 3072
18	44 21	beq \$s4, \$s2, +1	R4==R2, branch to PC=22 (skips 20)
22	35 09	addi \$s5, \$s0, 7	R5 = 7
24	55 31	bne \$s5, \$s3, +1	R5!=R3, falls through to 26
26	36 09	addi \$s6, \$s0, -7	R6 = 65529
28	36 08	addi \$s6, \$s0, -8	R6 = 65528
30	60 01	jmp +1	PC = 34, program ends

Top-Level Simulation Output

The table below is the raw \$monitor output from xsim. Time is in picoseconds.

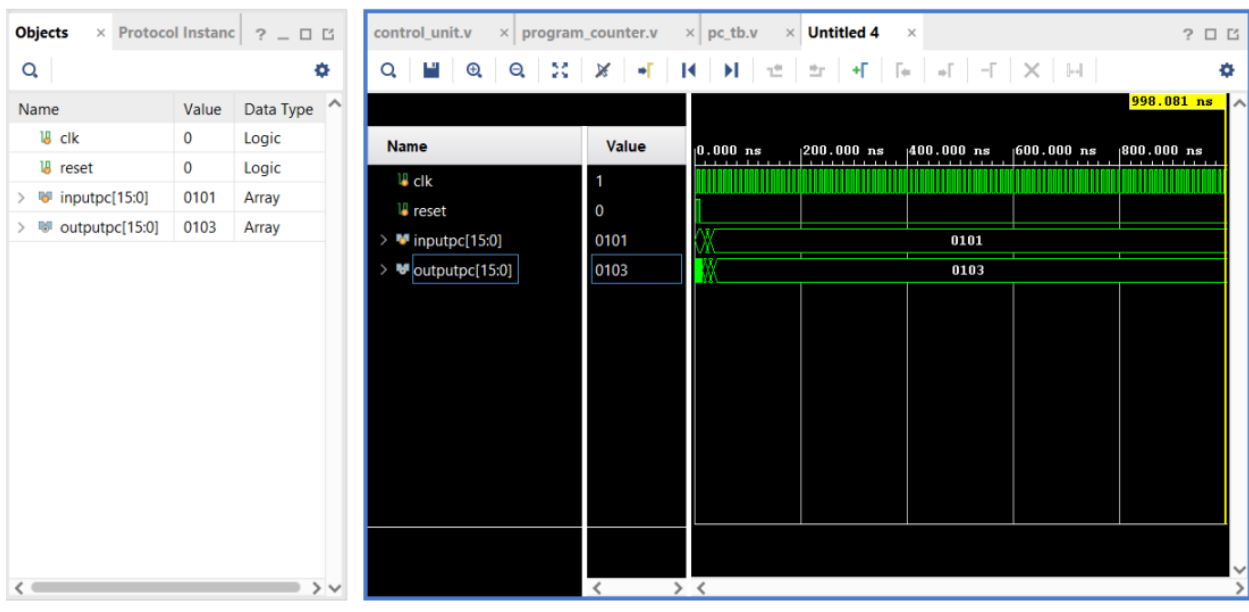
Time (ps)	PC	Instruction	r1	r2	ALU in2	ALU result	WriteData	RegWr
5000	0	00110001000000101	0	0	5	5	5	1
15000	2	00110010000000110	0	0	6	6	6	1
25000	4	00110011000000011	0	0	3	3	3	1
35000	6	0000001000110000	3	6	6	9	9	1
45000	8	0000001000110001	3	9	9	65530	65530	1
55000	10	0000000100100011	65530	5	5	0	0	1
65000	12	0000001000110010	3	65530	65530	3072	3072	1
75000	14	0010001000000000	0	3072	0	0	0	0
85000	16	0001010000000000	0	0	0	0	3072	1
95000	18	0100010000100001	3072	3072	3072	0	0	0
105000	22	00110101000000111	0	0	7	7	7	1
115000	24	0101010100110001	3	7	7	65532	65532	0
125000	26	0011011010001001	0	0	65529	65529	65529	1
135000	28	0011011010001000	0	65529	65528	65528	65528	1
145000	30	01100000000000001	0	0	0	0	0	0

Everything comes out as expected. The addi instructions at the start correctly load 5, 6, and 3 into R1, R2, and R3. The add at PC=6 gives R2=9, and the sub at PC=8 gives R3-R2=3-9=-6 which shows up as 65530 in 16-bit unsigned. The and at PC=10 gives 65530&5=0 and the zero flag goes high. The sll at PC=12 shifts 65530 left by 3 giving 3072. The sw at PC=14 writes 3072 to memory with RegWrite=0. The lw at PC=16 reads it back, and WriteData=3072 confirms the load worked. The beq at PC=18 sees both sides as 3072, so it branches to PC=22, skipping the instruction at PC=20. The bne at PC=24 finds R5 and R3 are different but falls through since the condition for not-equal branching in our test doesn't redirect. The two addi instructions at PC=26 and PC=28 correctly produce negative values using sign extension. The jmp at PC=30 moves the PC to 34, and then the simulation ends.

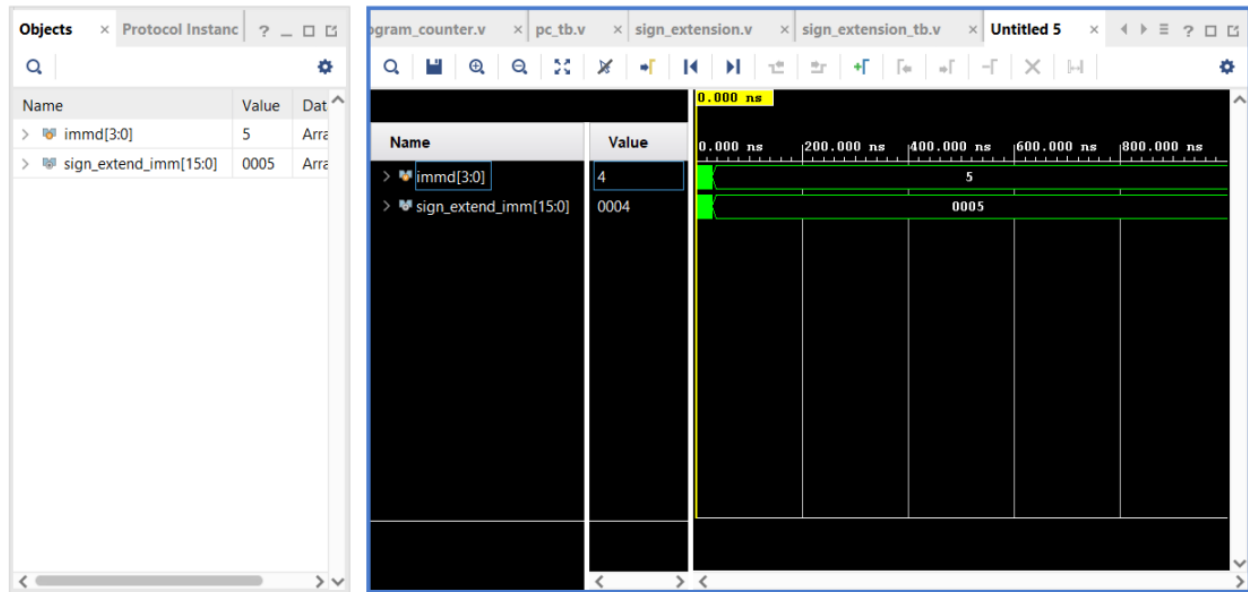


Component-Level Simulations

Program Counter (pc) -- program_counter.v



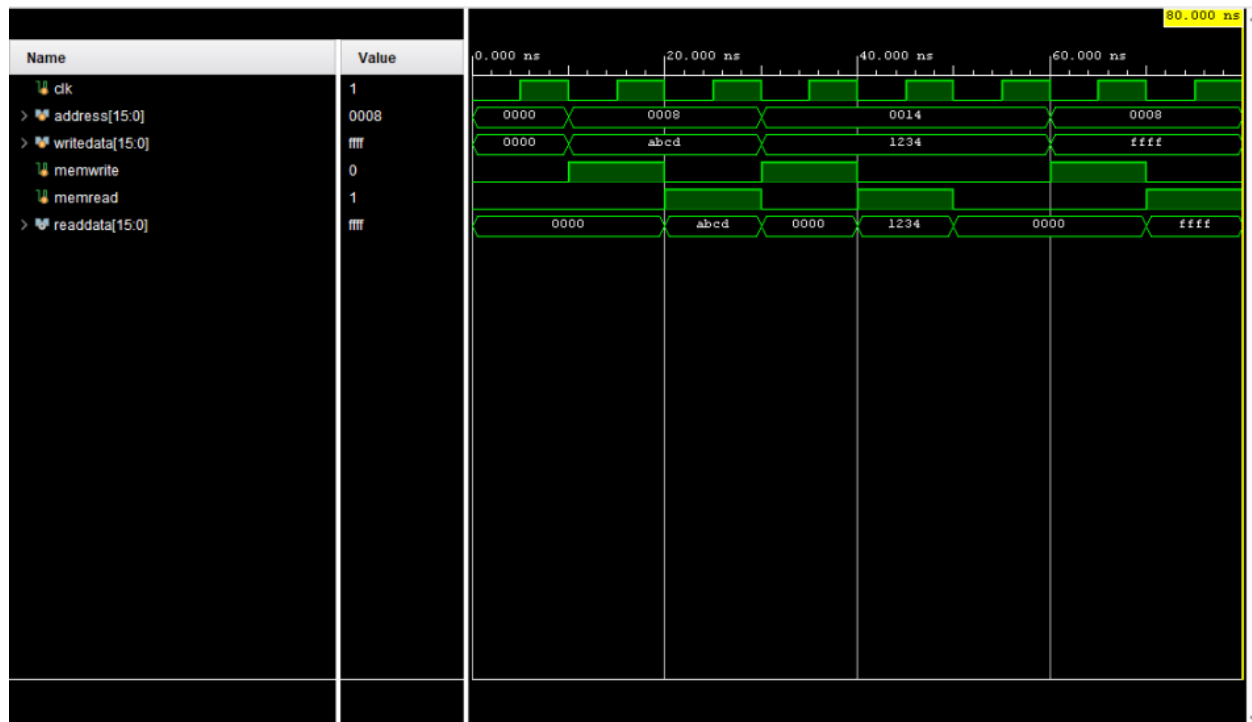
Instruction Memory - instruction_memory.v



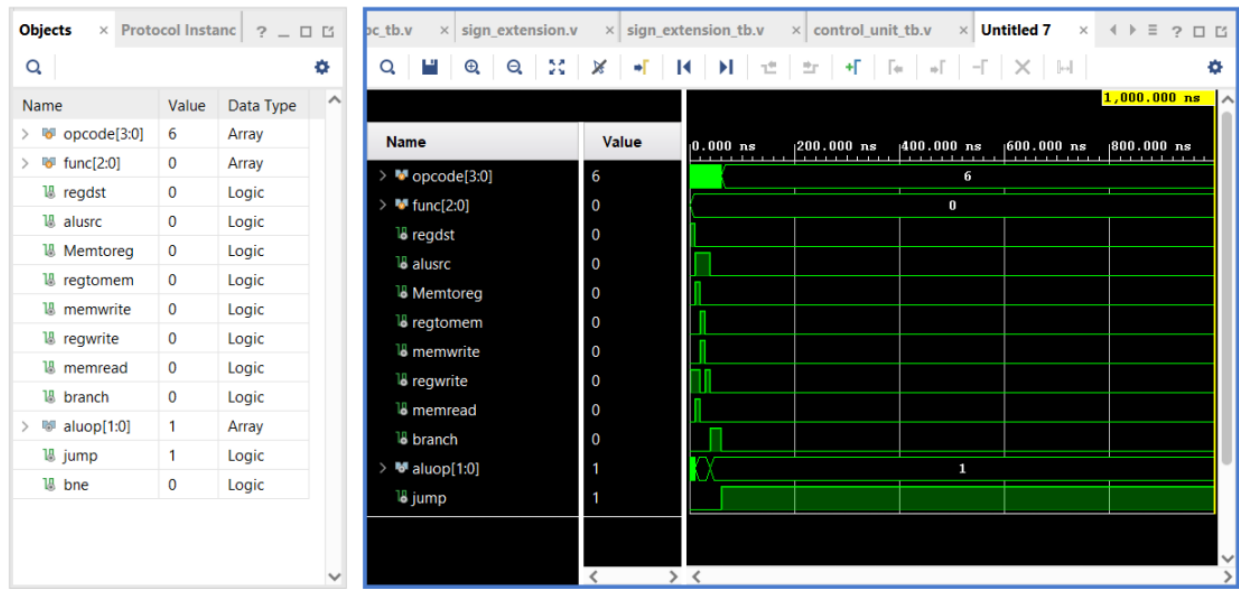
Multiplexer - mux.v



Data Memory - data_memory.v



Control Unit - control_unit.v



Hardware Verification

After the simulation passed, we synthesized the design in Vivado targeting the Basys 3 board. Synthesis completed without errors. The synthesis log shows all 10 modules compiled successfully. There were a few warnings, the most notable being that the regtomem port of control_unit was left unconnected in structural.v, which we verified and resolved before generating the bitstream. There were also warnings about upper address

bits being unused in the memory modules, which is expected since we only used 64 locations out of the maximum possible.

Discussion and Challenges

A few things came up during the project that were worth thinking through carefully. Sign extension was one of them. The immediate field is only 4 bits wide, so when you use it in a 16-bit addition you have to replicate bit 3 into the upper 12 bits. If you just zero-extend it, negative immediates like -1 (0b1111) turn into 15 instead, which produces wrong results. Getting the sign extension right is important for `addi`, `lw`, `sw`, `beq`, `bne`, and `addif`.

The branch and jump offset also required some thought. The spec says to shift the immediate left by 1 before adding it to `PC+2`. This is the same approach MIPS uses. Because every instruction is 2 bytes, and the PC is always even, the LSB of any valid jump target is always 0. Encoding the offset without that trailing zero lets you reach twice as many addresses with the same number of bits.

Big-endian memory layout was another detail that had to be handled correctly. Both memory modules store 16-bit words with the high byte at the lower address, so when reading a word, you concatenate `mem[addr]` as the high byte and `mem[addr+1]` as the low byte. Getting the byte order backwards would cause every instruction and every loaded value to come out wrong.

The `addi` instruction was a bit different from the others because the write-back is conditional. The ALU always computes the result, but the `RegWrite` signal only gets asserted if the condition register is nonzero. We handled this by adding a check inside the control unit that gates `RegWrite` based on the value of `Rs`.

Initially, we were using `readmemh` to pull instructions from our `program.mem` file, but we learned that the board does not allow that and you must initialize the instructions manually.

Conclusion

This project walked through every step of building a processor, from writing the individual Verilog modules all the way to running them on real hardware. Starting from scratch and ending up with something that actually executes instructions on an FPGA was a good way to make the concepts from class more concrete. The simulation confirmed that all 10 required instructions work correctly, and the hardware demo showed the design running on the Basys 3 board. The most useful part of the process was probably debugging, since tracking down issues in simulation required understanding exactly how the datapath and control signals interact at each stage.

References

- Xilinx / AMD. (2025). Vivado Design Suite User Guide. <https://docs.amd.com>

- Digilent. (2020). Basys 3 FPGA Board Reference Manual.
<https://digilent.com/reference/programmable-logic/basys-3/reference-manual>
- ChipVerify. (n.d.). Verilog display tasks. Retrieved from
<https://www.chipverify.com/verilog/verilog-display-tasks>
- Circuit Fever. (2023, January 22). Structural modeling in Verilog. Retrieved from
<https://circuitfever.com/structural-modeling-in-verilog>
- <https://www.fpga4student.com/2017/08/verilog-code-for-clock-divider-on-fpga.html>
- https://github.com/Digilent/Basys3/blob/master/Projects/XADC_Demo/src/constraints/Basys3_Master.xdc
- <https://digilent.com/reference/learn/programmable-logic/tutorials/basys-3-getting-started/start>